

Scritto 5 Giugno 2026

Esercizio 1

Una relazione su interi è un insieme di coppie (m,n) dove m ed n sono due valori interi. Si considerino le seguenti strutture:

```
1 struct coppia { int first ; int second ;} ;
2 struct relation { int num_coppie; coppia REL[length] ; } ;
```

dove `length` è una costante. Una relazione R è implementata come valore di tipo `relation` dove i primi `num_pairs` dell'array `REL` contengono le coppie in R (ci possono essere anche duplicati; la coppia (m,n) è diversa da (n,m));

(A) Definire la funzione

```
1 bool sub_relation(relation R, relation S)
```

che ritorna `true` se ogni coppia in R si trova anche in S , `false` altrimenti (non si considera il numero di occorrenze per i duplicati).

(B) Definire la funzione

```
1 int cardinalita(relation R)
```

che ritorna il numero di coppie differenti in R .

E' possibile usare funzioni ausiliarie, se definite

Codice base fornito:

```
1 const int length = 100 ;
2
3 struct coppia { int first ; int second ;} ;
4
5 struct relation { int num_coppie; coppia REL[length] ; } ;
6
7 bool sub_relation(relation R, relation S) ;
8
9 int cardinalita(relation R) ;
```

Esercizio 2

La seguente struttura dati implementa una lista di smartphone:

```
1 struct smartp{ char model[20]; char prod[20]; int price; smartp *next ;} ;
2 typedef smartp *psmartp ;
```

dove, per ogni smartphone, si memorizzano modello, produttore e prezzo.

La lista è ordinata per produttore (in ordine alfabetico) e, per ogni produttore, in ordine decrescente di prezzo. Saranno quindi memorizzati, ad esempio, prima tutti gli smartphone Apple e poi tutti i Samsung. Gli smartphone Apple poi saranno ordinati dal più caro al più economico.

Definire le funzioni:

(A)

```
1 int give_number(psmartp C, int top)
```

che prende in input una lista C e un prezzo top e ritorna il numero di produttori che vendono almeno un modello il cui prezzo è minore di top.

(B)

```
1 psmartp add(psmartp C, char M[], char P[], int c)
```

che aggiunge uno smartphone di modello M, produttore P e costo c nella posizione corretta della lista C. La funzione ritorna la lista modificata.

Codice base fornito:

```
1 struct smartp{ char model[20]; char prod[20]; int price; smartp *next ;} ;
2 typedef smartp *psmartp ;
3
4 int give_number(psmartp C, int top) ;
5
6 psmartp add(psmartp C, char M[], char P[], int c) ;
```

Esercizio 3

Un cinema Multisala è composto da più sale e ciascuna sala è caratterizzata dal numero di posti disponibile, dal numero di posti prenotati e da un valore booleano che definisce se si tratta di una sala base o 3D. La Multisala è caratterizzato dal prezzo del biglietto per la sala base e da un prezzo per le sale 3D.

Si implementi la

-- class MultiSala e il relativo costruttore

```
1 Multisala(double prezzo_base, double prezzo_3D, int capienza[], bool is3D[])
```

dove capienza e un array di 10 interi che determinano la capienza max delle relative sale, mentre is3D e un array di 10 booleani che specificano se una sale e` di base (valore = false) oppure 3D (valore = true). Si assumi che la multisala abbia esattamente 10 sale;

-- il metodo `bool prenota_posti(int n, int p)` , il quale riceve in input il numero di sala n, controlla che ci sia disponibilit  di p posti e, se possibile, incrementa il numero di posti prenotati per quella sala e ritorna true; ritorna false se non c'e disponibilit  di posti o la sala e` sbagliata (i numeri sono da 0 a 9);

-- il metodo `int conta_tickets(bool t)` , il quale riceve un parametro booleano t che indica se contare il totale dei biglietti delle sale base (t = false) o delle sale 3D (t = true). Ritorna il numero totale di biglietti del tipo t venduti nel Cinema;

-- il metodo `double incasso_totale()` , il quale calcola (e restituisce) il totale dell'incasso, ottenuto come la somma dei biglietti su tutte le sale (tenendo conto del tipo di biglietto).

Una Multisala pu  anche offrire un ristorante che   caratterizzato dal numero di posti massimo, numero di posti prenotati e dal prezzo di un unico menu fisso che   possibile acquistare. Si implementi

-- la class MultiSalaPlus come sottoclasse di MultiSala, e il relativo costruttore con tipo

```
1 MultiSalaPlus(double pr, double pr3D, int n[], bool t[], int nRisto, double prRisto)
```

(assumere che ci sia un solo ristorante: nRisto indica il numero di posti e prRisto il prezzo del menu`);

-- il metodo `bool prenota_postiRist(int p)` , il quale riceve in input il numero di posti da prenotare p, controlla che ci sia disponibilit  di p posti e, se possibile, incrementa il numero di posti prenotati e ritorna true; ritorna false se non c'e disponibilit  di posti;

-- il metodo `double incasso_totale()` , opportunamente modificato per includere i ricavi derivanti dal ristorante. Si sfrutti opportunamente l'ereditiet .

Codice base fornito:

```
1  struct sala{
2      int posti_totali;
3      int posti_prenotati;
4      bool is3D;
5  };
6
7  struct ristorante{
8      int totale_consumazioni;
9      int consumazioni_prenotate ;
10 };
11
12 class MultiSala{
13 protected:
14     ...
15 public:
16     MultiSala(double pr, double pr3D, int num_posti[], bool tipo_sala[])
17
18     bool prenota_posti(int n, int p)
19
20     int conta_tickets(bool t)
21
22     double incasso_totale()
23 };
24
25
26 class MultiSalaPlus ... {
27 protected:
28     ...
29 public:
30     MultiSalaPlus(double pr, double pr3D, int n[], bool t[], int nRisto, double prRisto)
31
32     bool prenota_postiRist(int p)
33
34     double incasso_totale()
35
36 };
```